

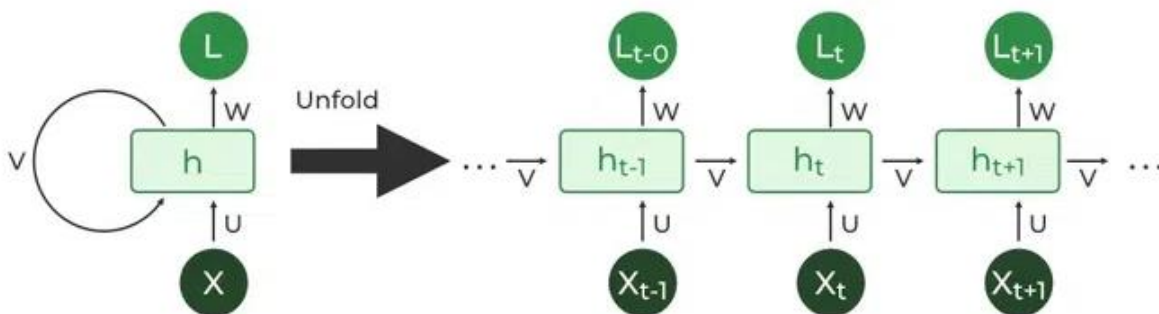
ANN and DL (CAI-361)

Objectives:

- Recurrent Neural Networks
- Backpropagation Through Time (BPTT)

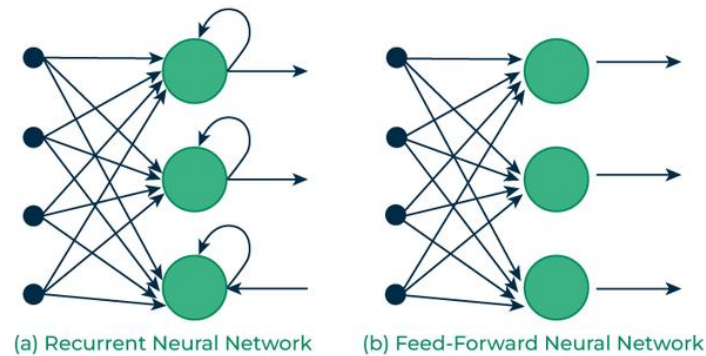
Recurrent Neural Network:

Recurrent Neural Network (RNN) is a type of Neural Network where the output from the previous step is fed as input to the current step. In traditional neural networks, all the inputs and outputs are independent of each other. Still, in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words. Thus RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is its **Hidden state**, which remembers some information about a sequence. The state is also referred to as **Memory State** since it remembers the previous input to the network. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output.



How RNN differs from Feedforward Neural Network?

ANN that do not have looping nodes are called feed forward neural networks. Because all information is only passed forward. These networks are appropriate for image classification tasks, for example, where input and output are independent. Nevertheless, their inability to retain previous inputs automatically renders them less useful for sequential data analysis. The fundamental processing unit in a Recurrent Neural Network (RNN) is a Recurrent Unit, which is not explicitly called a “**Recurrent Neuron.**” This unit has the unique ability to maintain a hidden state, allowing the network to capture sequential dependencies by remembering previous inputs while processing.

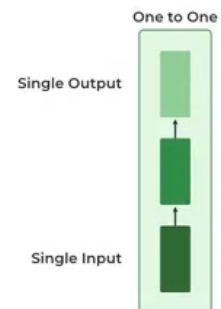


Types Of RNN

There are four types of RNNs based on the number of inputs and outputs in the network.

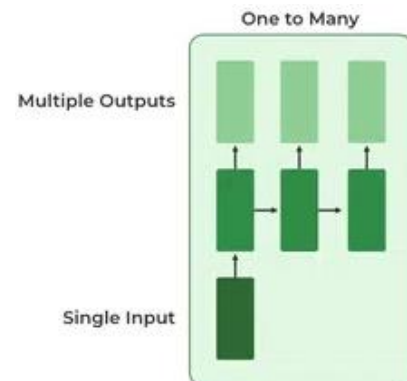
1. One to One

This type of RNN behaves the same as any simple Neural network it is also known as Vanilla Neural Network. In this Neural network, there is only one input and one output.



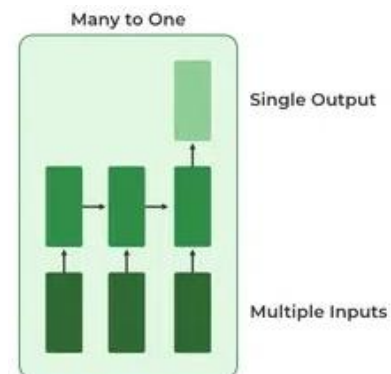
2. One To Many

In this type of RNN, there is one input and many outputs associated with it. One of the most used examples of this network is Image captioning where given an image we predict a sentence having Multiple words.



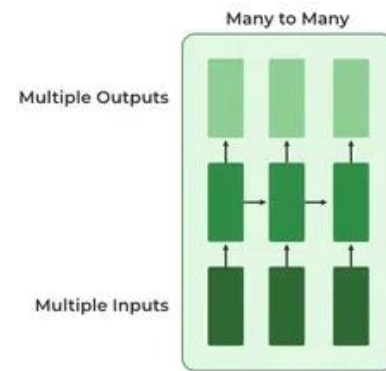
3. Many to One

In this type of network, Many inputs are fed to the network at several states of the network generating only one output. This type of network is used in the problems like sentimental analysis. Where we give multiple words as input and predict only the sentiment of the sentence as output.



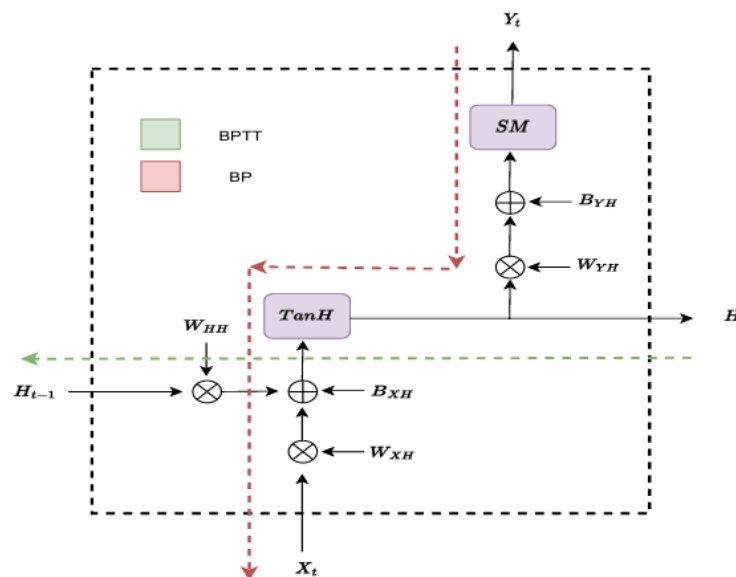
4. Many to Many

In this type of neural network, there are multiple inputs and multiple outputs corresponding to a problem. One Example of this Problem will be language translation. In language translation, we provide multiple words from one language as input and predict multiple words from the second language as output.



Recurrent Neural Network Architecture

RNNs have the same input and output architecture as any other deep neural architecture. However, differences arise in the way information flows from input to output. Unlike DNN in RNN, the weight across the network remains the same. It calculates hidden state $\mathbf{h}_i$ for every input $\mathbf{x}_i$. Inside architecture of a recurrent unit is given below



The Recurrent Neural Network consists of multiple fixed activation function units, one for each time step. Each unit has an internal state which is called the **hidden state** of the unit. This hidden state signifies the past knowledge that the network currently holds at a given time step. This hidden state is updated at every time step to signify the change in the knowledge of the network about the past. The hidden state is updated using the following recurrence relation:-

$$h_t = f(h_{t-1}, x_t)$$

where,

- $h_t \rightarrow$ current state
- $h_{t-1} \rightarrow$ previous state
- $x_t \rightarrow$ input state

Formula for applying Activation function(tanh)

$$h_t = \tanh (W_{hh} h_{t-1} + W_{xh} x_t + B_{xh})$$

where,

- $W_{hh} \rightarrow$ weight at recurrent neuron
- $W_{xh} \rightarrow$ weight at input neuron
- $B_{xh} \rightarrow$ bias term

Formula for calculating output:

$$Y_t = W_{yh} h_t + B_{yh}$$

- $Y_t \rightarrow$ output
- $W_{yh} \rightarrow$ weight at output layer

These parameters are updated using Backpropagation. However, since RNN works on sequential data here we use an updated backpropagation which is known as Backpropagation through time.

Backpropagation Through Time (BPTT)

In RNN the neural network is in an ordered fashion and since in the ordered network each variable is computed one at a time in a specified order like first h_1 then h_2 then h_3 so on. Hence we will apply backpropagation throughout all these hidden time states sequentially.

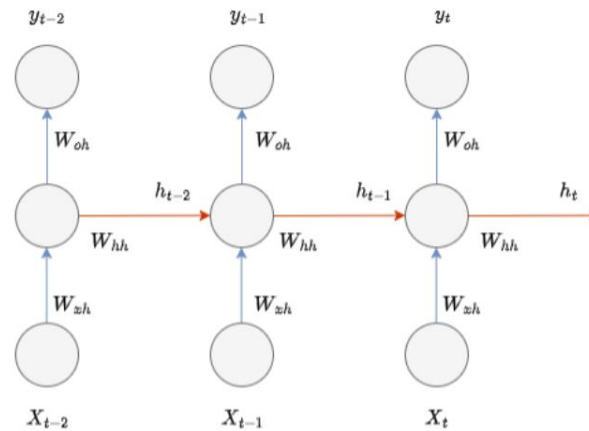
To figure out how to derive BPTT in RNN, the first thing we need to do is to choose a Loss function, Here I'm choosing the **Mean Squared Error (MSE)** Loss, below is the formula for calculating the MSE loss function

$$L = \frac{1}{N} \sum_{t=1}^N (y - y_t)^2$$

For propagating the loss backward, we need to find the derivative of loss, the derivative of MSE is simply.

$$\frac{\partial L}{\partial y_t} = \frac{2}{n} \sum_{t=t}^n (y - y_t)$$

The objective of BPTT is to find the optimal weights and biases in each update in our RNN, i.e. we need to update weights using gradient descent, there are three categories of weights, the weights connecting inputs **W_{xh}** , the weights through hidden states **W_{hh}** , and the output weights **W_{oh}** .

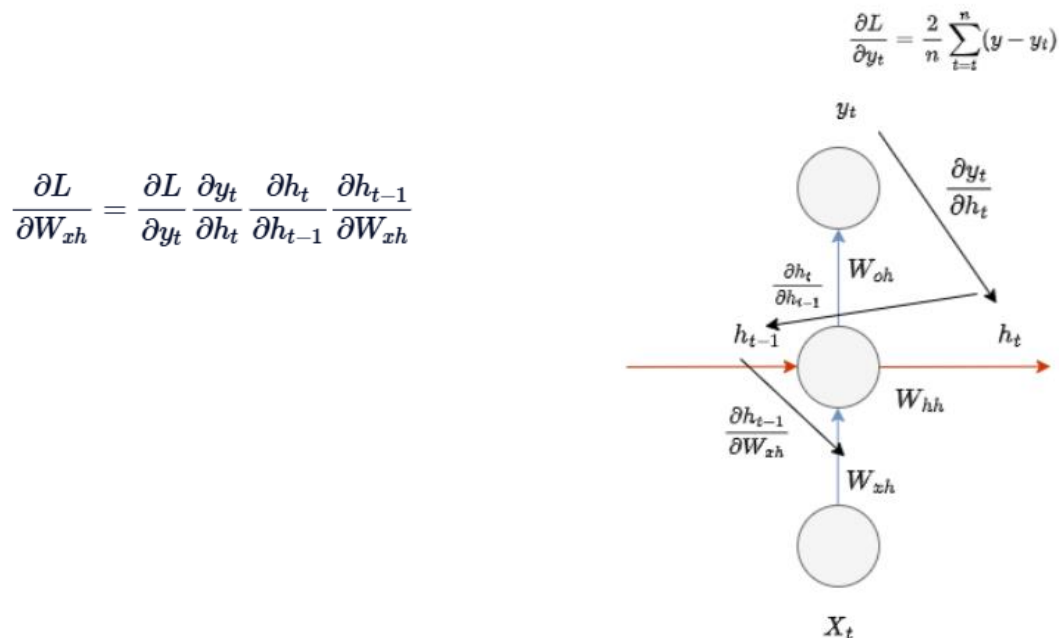


Derivative of Loss with respect to Weights W_{xh} , W_{hh} , and, W_{oh}

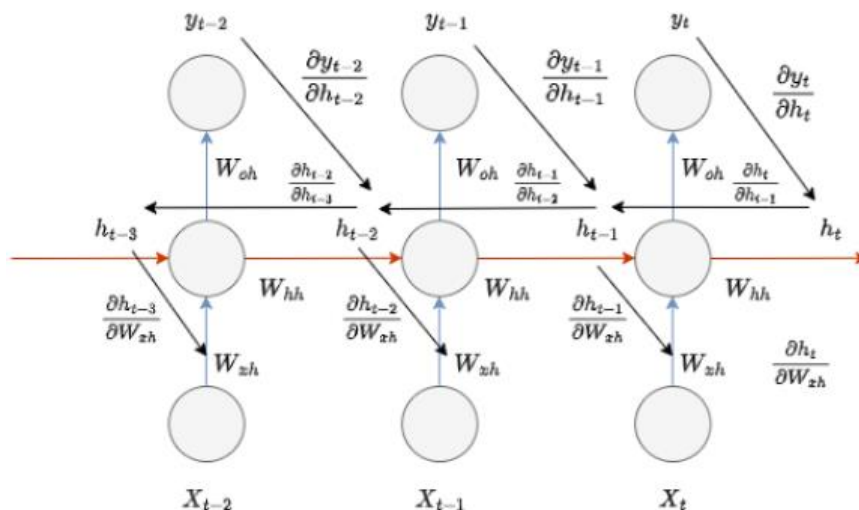
Think what are the terms that affect the change in weights $\mathbf{W}$ when considering a single time step?

- Obviously, we can say the change in loss with respect to the change in output $\frac{\partial L}{\partial y_t}$ is influencing weights since weights are a part of the output produced, Similarly,
- The change in the output with respect to the change in hidden states $\frac{\partial y_t}{\partial h_t}$ is influencing weights,
- Then the change in hidden states to the change in previous hidden states $\frac{\partial h_t}{\partial h_{t-1}}$ is influencing weights, and
- Finally, the change in previous hidden states with respect to the change in Weights $\frac{\partial h_{t-1}}{\partial W}$.

This is the **chain rule** which tells how much the weights $\mathbf{W}$ changes with respect to values related to it. Let's consider $\mathbf{W}_{xh}$, which is the weights connecting the input layer and hidden layer, and see how the gradients are passed through the network.



Here's how to calculate the gradient of loss in relation to input weights for a single time step. However, when dealing with multiple time steps, the gradients of each time step should be summed up. But beyond this, due to the interconnected nature of hidden states across time, it's crucial to accumulate the gradients of these hidden states as well. For this, you need to multiply the gradients of hidden states together in each time step.



It seems like a lot of gibberish out there above, but this is how gradients flow through a Recurrent Neural Network. Let's see how we can compute one by one, Considering single time step,

$$\frac{\partial L}{\partial W_{xh}} = \frac{\partial L}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W_{xh}}$$

Considering 2 time steps

$$\frac{\partial L}{\partial W_{xh}} = \frac{\partial L}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W_{xh}} + \frac{\partial L}{\partial y_{t-1}} \frac{\partial y_{t-1}}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{\partial h_{t-2}}{\partial W_{xh}}$$

And three time steps

$$\frac{\partial L}{\partial W_{xh}} = \frac{\partial L}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W_{xh}} + \frac{\partial L}{\partial y_{t-1}} \frac{\partial y_{t-1}}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{\partial h_{t-2}}{\partial W_{xh}} + \frac{\partial L}{\partial y_{t-2}} \frac{\partial y_{t-2}}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{\partial h_{t-2}}{\partial h_{t-3}} \frac{\partial h_{t-3}}{\partial W_{xh}}$$

As time steps increase the hidden state gradient is accumulating. Now consider thousands of time steps, in that case, the gradients of hidden states (values in the center of the equation) grow over and over again in each time step, unfortunately, we cannot write it, so let's represent this in the general formula, here is how to generalize it,

$$\frac{\partial L}{\partial W_{xh}} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \frac{\partial y_{t-i}}{\partial h_{t-i}} \left(\prod_{j=t-i+1}^t \frac{\partial h_{t-j+1}}{\partial h_{t-j}} \right) \frac{\partial h_{t-i-1}}{\partial W_{xh}}$$

Where, y_{t-i} represents the output at time step $t-i$

h_{t-i} represents the hidden state at time step $t-i$

Now this symbol $\prod$ is the product that is used to multiply a sequence of values, in our case, it is used to accumulate the gradients of hidden states through time.

Let's find the derivative of loss with respect to weights in the hidden units, It is so simple, All we need to do is to substitute W_{hh} in the place of W_{xh} in the equation,

$$\frac{\partial L}{\partial W_{hh}} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \frac{\partial y_{t-i}}{\partial h_{t-i}} \left(\prod_{j=t-i+1}^t \frac{\partial h_{t-j+1}}{\partial h_{t-j}} \right) \frac{\partial h_{t-i-1}}{\partial W_{hh}}$$

The idea is the same except you find the derivatives with respect to W_{hh} . This can be a little different when comes to the output layer, for finding W_{oh} , you just only need to consider the output y_t and Weights W_{oh} , so it can be written as,

$$\frac{\partial L}{\partial W_{oh}} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \frac{\partial y_{t-i}}{\partial W_{oh}}$$

Derivative of Loss With Respect to Bias b_h and b_o

We need to find the derivatives of bias in the hidden layer b_h and the bias in the output layer b_o , let's see how it can be done in the case of b_h ,

$$\frac{\partial L}{\partial b_h} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \cdot \frac{\partial y_{t-i}}{\partial h_{t-i}} \cdot \frac{\partial h_{t-i}}{\partial b_h}$$

Now for bias in the output layer, b_o will be,

$$\frac{\partial L}{\partial b_o} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \cdot \frac{\partial y_{t-i}}{\partial b_o}$$

The process of calculating derivatives with respect to bias terms in recurrent neural networks (RNNs) is similar to the backpropagation process in feed-forward neural networks, with the added consideration of hidden states due to the recurrent nature of RNNs.

Derivative of Loss With Respect to Weights in Input Layer W_{xh} ,

$$\frac{\partial L}{\partial W_{xh}} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \frac{\partial y_{t-i}}{\partial h_{t-i}} \left(\prod_{j=t-i+1}^t \frac{\partial h_{t-j+1}}{\partial h_{t-j}} \right) \frac{\partial h_{t-i-1}}{\partial W_{xh}}$$

Derivative of Loss With Respect to Weights in the Hidden States W_{hh} ,

$$\frac{\partial L}{\partial W_{hh}} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \frac{\partial y_{t-i}}{\partial h_{t-i}} \left(\prod_{j=t-i+1}^t \frac{\partial h_{t-j+1}}{\partial h_{t-j}} \right) \frac{\partial h_{t-i-1}}{\partial W_{hh}}$$

Derivative of Loss With Respect to Weights in the Output Layer W_{oh} ,

$$\frac{\partial L}{\partial W_{oh}} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \frac{\partial y_{t-i}}{\partial h_{t-i}} \left(\prod_{j=t-i+1}^t \frac{\partial h_{t-j+1}}{\partial h_{t-j}} \right) \frac{\partial h_{t-i-1}}{\partial W_{oh}}$$

Derivative of Loss With Respect to Bias in the Hidden States b_h ,

$$\frac{\partial L}{\partial b_h} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \cdot \frac{\partial y_{t-i}}{\partial h_{t-i}} \cdot \frac{\partial h_{t-i}}{\partial b_h}$$

Derivative of Loss With Respect to Bias in the Output layer b_o ,

$$\frac{\partial L}{\partial b_o} = \sum_{i=0}^t \frac{\partial L}{\partial y_{t-i}} \cdot \frac{\partial y_{t-i}}{\partial b_o}$$

Updating Weights and Bias Using Gradient Descent

The update rule is quite similar across different types of neural networks, including recurrent neural networks (RNNs), with some variations depending on the optimization algorithm used. Since we are using Gradient Descent, the update rule will be,

$$\text{Updating } \mathbf{W}_{ho} \quad W_{oh} \leftarrow W_{oh} - \alpha \cdot \frac{\partial L}{\partial W_{oh}}$$

$$\text{Updating } \mathbf{W}_{xh} \quad W_{xh} \leftarrow W_{xh} - \alpha \cdot \frac{\partial L}{\partial W_{xh}}$$

$$\text{Updating } \mathbf{W}_{hh} \quad W_{hh} \leftarrow W_{hh} - \alpha \cdot \frac{\partial L}{\partial W_{hh}}$$

$$\text{Updating } \mathbf{b}_h \quad b_h \leftarrow b_h - \alpha \cdot \frac{\partial L}{\partial b_h}$$

$$\text{Updating } \mathbf{b}_o \quad b_o \leftarrow b_o - \alpha \cdot \frac{\partial L}{\partial b_o}$$

Advantages of RNN:

Recurrent Neural Networks (RNNs) have several advantages that make them particularly effective for tasks involving sequential data:

1. **Effective in Sequence Modeling Tasks:** RNNs are specifically designed to handle data where the order of inputs matters, such as in natural language processing (NLP), time series analysis, and speech recognition. Their architecture allows them to process sequences of data in a way that captures temporal dependencies.
2. **Memory that Remembers Contextual Information:** RNNs maintain hidden states that serve as memory cells, allowing them to remember information from previous inputs. This capability enables RNNs to capture context and dependencies over time, which is crucial for understanding sequences where the meaning can change based on prior elements.
3. **Variable Length Input Processing:** Unlike traditional neural networks that require fixed-size input vectors, RNNs can process sequences of varying lengths. This flexibility is essential for applications like language modeling, where sentences can vary significantly in length.

4. **Parameter Sharing:** RNNs utilize the same weights across different time steps in the sequence. This means that the model size does not increase with the length of the sequence, which leads to more efficient memory usage and computational requirements. This parameter sharing allows RNNs to generalize better across different parts of the sequence.

Overall, these advantages make RNNs a powerful tool for tasks that involve understanding and generating sequential data.

Disadvantages of RNN

Recurrent Neural Networks (RNNs) have several disadvantages that can limit their effectiveness in certain applications:

1. **Vanishing Gradient Problem:** This is a significant issue where the gradients used in training become very small as they are backpropagated through many time steps. When this happens, the network struggles to learn from earlier time steps, making it difficult to capture long-range dependencies in the data. As a result, RNNs often fail to remember important information from the past, which is crucial for tasks like language modeling or time series prediction.
2. **Exploding Gradient Problem:** In contrast to the vanishing gradient problem, the exploding gradient problem occurs when gradients become excessively large during training. This can lead to numerical instability, causing the model's weights to diverge and making it difficult to train effectively. While this issue is easier to detect and manage (for instance, through techniques like gradient clipping), it still poses a challenge for RNNs.
3. **Limited Memory:** Traditional RNNs have a limited capacity to retain information over long sequences. As the sequence length increases, the network may "forget" important information from earlier time steps, which can be particularly problematic in tasks that require understanding context from distant past inputs.
4. **Biased Toward Recent Data:** Because RNNs process sequences in order, they tend to give more weight to recent inputs while diminishing the influence of earlier data. This bias can hinder the network's ability to capture global context, making it less effective in scenarios where understanding the entire sequence is essential.

5. **Difficulty with Parallelization:** RNNs inherently process data sequentially, meaning that each time step relies on the output from the previous one. This sequential nature makes it challenging to parallelize the training process, which can lead to longer training times. In contrast, modern hardware architectures like GPUs are optimized for parallel processing, and RNNs do not fully leverage this capability, resulting in inefficiencies.

Overall, while RNNs are powerful for certain tasks, these disadvantages have led to the development of alternative architectures, such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), which address some of these issues